When the expected condition is met, obtain this web element.

```
WebDriverWait wait = new WebDriverWait(getDriver(),5);
WebElement typeaheadList =
    wait.until(ExpectedConditions.presenceOfElementLocated(
                By.className(".typeahead")));
```

Wait for up to five seconds.

Wait until this element is present on the page.

These predefined conditions cover many common situations. But occasionally you'll need to do something more specific. Again, WebDriver offers several options. The `FluentWait` class allows you to create arbitrary wait parameters on the fly using a readable fluent API:

```
Wait<WebDriver> wait = new FluentWait<WebDriver>(getDriver())
                       .withTimeout(30, TimeUnit.SECONDS)
                          .pollingEvery(1, TimeUnit.SECONDS)
                          .ignoring(NoSuchElementException.class);
```

Wait for up to 30 seconds.

Don't fail if an element isn't found yet.

Check the page every second.

You can use this `wait` object with one of the predefined conditions from the `ExpectedConditions` class, or you can write your own condition. A condition takes the form of a `Function` object (from the Google Guava library, https://code.google.com/p/guava-libraries/), and it typically returns either a `WebElement` (if you're waiting for a web element to become available) or a `Boolean` (if you're waiting for some more general condition). In the following example, you wait until the type-ahead list is present on the page and contains entries:

```
wait.until(typeaheadIsNotEmpty());
...
private Function<WebDriver, Boolean> typeaheadIsNotEmpty(){
  return new Function<WebDriver, Boolean>() {
    public Boolean apply(WebDriver driver){
      List<WebElement> typeaheadItems
               = driver.findElements(By.cssSelector(".typeahead li"));
      return typeaheadItems.size() > 0;
    }
  };
}
```

❶ Wait until the type-ahead list is both present and contains entries.

❷ This function takes a WebDriver instance and returns a Boolean.

All Function implementations must implement the apply() method. ❸

❹ Return true if the type-ahead list is present and isn't empty.

In this code sample, you create your own implementation of the `Function` interface to check the presence and size of the type-ahead list. The `Function` interface here represents a function that takes a `WebDriver` instance and returns a `Boolean` value ❷. When this `Boolean` value is `true`, the test will be able to continue ❶. The `Function` interface defines the `apply()` function ❸, which you need to implement. In this case, the condition is relatively simple: the implementation retrieves and checks the size of the type-ahead list ❹. This implementation could be more complicated if you need to check more involved conditions.

8.2.5 Writing test-friendly web applications

The style and quality of your web application code has a significant influence on how easy or hard it will be to test. Applications with clean HTML code, identifiers, names, and CSS classes for all the significant elements on a page make testing easier